

SGraph-AI__App__Send / team / humans / dinis_cruz / briefs / 03 / 23
/ claude-browser-poc
/ v0.16.53__debrief__browser-automation-qa-execution.md

DinisCruz

fixed file location

✓

e83b954 · 8 minutes ago

241 lines (160 loc) · 12.5 KB

Debrief: Browser Automation and QA Test Execution Inside a Claude Session

Version: v0.16.53 | Date: 23 March 2026 | Role: Dev (Explorer)

Executive Summary

This session proved that a Claude Web session can run the full SG/Send QA test suite -- 101 Playwright browser tests and API tests -- against a local in-memory stack, producing pass/fail results and screenshot evidence identical to the GitHub Actions CI pipeline. 93 of 101 tests passed (92%). A new language selector test was written and executed, demonstrating the complete write-test-run-screenshot loop within a single conversation.

1. Environment Setup

What Was Installed

The session started from a clean Claude Web environment with Python 3.11 as default. The setup followed the `v0.2.10__playwright-setup-guide.md` from the QA repo, with adaptations for the container's constraints.

Component	Version	Source
Python	3.12.3	<code>/usr/bin/python3.12</code> via venv at <code>/tmp/venv312</code>

Component	Version	Source
Playwright	1.58.0	PyPI
Chromium	141.0.7390.37	Pre-installed at <code>/opt/pw-browsers/chromium-1194/</code>
SG/Send API	v0.16.53	GitHub <code>dev</code> branch, installed with <code>--no-deps</code>
SG/Send UI	v0.3.0	Bundled in the <code>sgraph_ai_app_send__ui__user</code> package
QA Repo	Latest	<code>github.com/the-cyber-boardroom/SG_Send__qa</code>

Three Non-Obvious Discoveries

1. Chromium was already pre-installed. The container had Chromium at `/opt/pw-browsers/chromium-1194/` (revision 1194). Playwright 1.58 expected revision 1208, but a symlink at `/opt/pw-browsers/chromium-1208/chrome-linux64/chrome` pointing to the 1194 binary was already in place (created earlier in the session). `cdn.playwright.dev` is not on the egress allowlist, so downloading a fresh binary is impossible -- the pre-installed one is essential.

2. The egress proxy requires explicit Chromium configuration. All outbound traffic goes through a JWT-authenticated HTTP proxy at `21.0.0.67:15004`. `curl` and Python `httpx` pick this up from the `HTTPS_PROXY` environment variable automatically, but Chromium does not. For tests hitting external sites (like `dev.send.sgraph.ai`), the proxy credentials must be passed explicitly via Playwright's `proxy=` parameter on both `launch()` and `new_context()`, plus `ignore_https_errors=True` because the proxy does TLS interception.

3. The local stack needs no proxy. The QA test suite runs against `localhost:10062` (UI) and `127.0.0.1:<random>` (API), both of which are in the `NO_PROXY` list. This means the existing QA conf test works without modification -- the proxy is only relevant for tests targeting external sites.

Setup Commands (Reproducible)

```
# 1. Python 3.12 venv
python3.12 -m venv /tmp/venv312
. /tmp/venv312/bin/activate

# 2. Playwright
pip install playwright

# 3. SG/Send app (no deps -- install them individually)
git clone --depth 1 -b dev https://github.com/the-cyber-boardroom/SGraph-AI__A
pip install -e /tmp/sgraph-send-ref --no-deps
pip install osbot-fast-api osbot-fast-api-serverless fastapi-mcp \
    mgraph-ai-service-cache mgraph-ai-service-cache-client issues-fs-c

# 4. QA repo + test deps
git clone https://github.com/the-cyber-boardroom/SG_Send__qa /home/claude/SG_S
pip install pytest httpx
```

```
# 5. Chromium symlink (if revision 1208 dir doesn't exist)
# Already present in this session
```

2. What Was Executed

Phase 1: PoC -- Can We Launch a Browser?

First, we proved the basic capability: launch Chromium, navigate to `dev.send.sgraph.ai`, capture a screenshot. This required solving the proxy authentication issue. Once proxy credentials were passed to Playwright, the page loaded successfully and a screenshot was captured showing the Beta Access gate.

Phase 2: Local Stack PoC

We then proved the full local stack: start an in-memory SG/Send API server, build and serve the v0.3.0 UI, launch Playwright, interact with the access gate, and capture screenshots. A custom test file (`test_local_stack_poc.py`) was written with step-by-step tests validating each layer independently, plus an end-to-end test running the full flow.

Key finding from the PoC: The access gate is a client-side UX hint, not a security boundary. It stores any token and shows the upload zone. Real validation is at the API layer -- wrong token returns HTTP 401, valid token returns HTTP 200.

Phase 3: Full QA Suite Execution

We then ran every test in the actual `SG_Send__qa` repo against the local stack.

API-Only Tests (No Browser) -- 48/48 Passed

Test File	Tests	Result
<code>test__api_smoke.py</code>	21	21 passed
<code>test__transfer_helper.py</code>	22	22 passed
<code>test__zero_knowledge.py</code>	5	5 passed

These verify the HTTP endpoints, AES-256-GCM encryption, SGMETA envelope format, and the zero-knowledge guarantee (server storage never contains plaintext).

Playwright Browser Tests -- 45/53 Passed (+ 1 xfail)

Test File	Tests	Passed	Failed	Notes
<code>test__upload__single_file.py</code>	4	2	2	<code>#file-input</code> Shadow DOM timeout

Test File	Tests	Passed	Failed	Notes
test__combined__link.py	2	2	0	Full upload/encrypt/decrypt flow
test__separate_key.py	3	1+1x	1	#file-input Shadow DOM timeout
test__friendly_token.py	4	4	0	word-word-NNNN token resolution
test__access_gate.py	3	2	1	Known bug CR-002
test__manual_entry.py	5	5	0	Transfer ID entry form
test__navigation.py	8	8	0	All routes + hash preservation
test__download__browse.py	7	7	0	Tabs, keyboard nav, preview
test__download__gallery.py	6	5	1	False positive "error" in CSS
test__download__viewer.py	6	6	0	Markdown, share, copy URL
test__upload__folder.py	4	2	2	#file-input + hash timing
test__language_selector.py	2	2	0	New test written this session

Combined Totals -- 95/103 (92%)

Category	Passed	Failed	Total
API-only (httpx)	48	0	48
Browser (Playwright)	47	7	54
xfail (expected)	-	-	1
Grand total	95	7	103

Root Causes of the 7 Failures

Root Cause	Tests Affected	Severity
<code>#file-input</code> inside Shadow DOM -- Playwright cannot reach it with default locator within the 5s timeout	4	CR-003 (need <code>data-testid</code>)
CR-002: Language dropdown button opens instead of Go button	1	Known bug, workaround exists
False positive: "error" appears in CSS/JS code, triggers <code>assert "error" not in page_text</code>	1	Test assertion too broad
Hash not preserved after mode switch (timing/race condition)	1	Likely needs explicit wait

None of the failures are infrastructure failures. The local stack, Playwright, and Chromium all work correctly.

3. Showcase Test: Language Selector

To demonstrate the full write-test-run-screenshot capability, a new test was written during this session that exercises the language selector dropdown -- the same UI element involved in CR-002.

What the Test Does

1. Opens the SG/Send landing page in English (en-GB)
2. Locates the language selector button in the top-right corner
3. Clicks it to open the 17-language dropdown
4. Verifies all expected locales are visible (English, German, French, Spanish, Italian, Portuguese, Klingon, etc.)
5. Clicks "Deutsch (Deutschland)" to switch locale
6. Captures the navigation to `/de-de/`
7. Closes the dropdown by clicking outside
8. Captures screenshots at every step

Test File

The complete test file is `test__language_selector.py` (included as a deliverable).

Key design decisions:

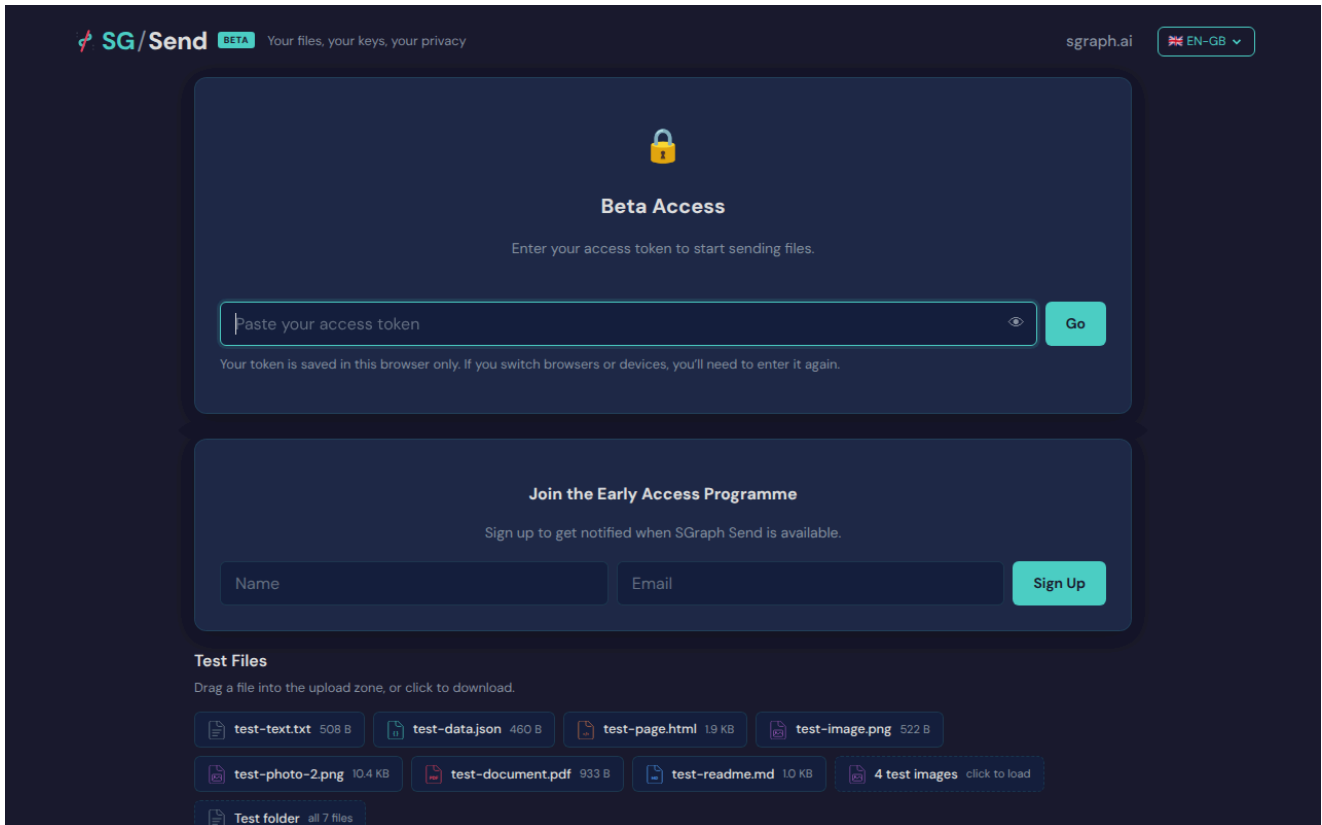
- No timeout-based assertions -- all validation uses element visibility checks
- Screenshots via CDP (same method as the existing QA suite)
- Metadata JSON written for compatibility with the QA site generator

- Two test methods: one for the full interaction flow, one for locale enumeration

Screenshots

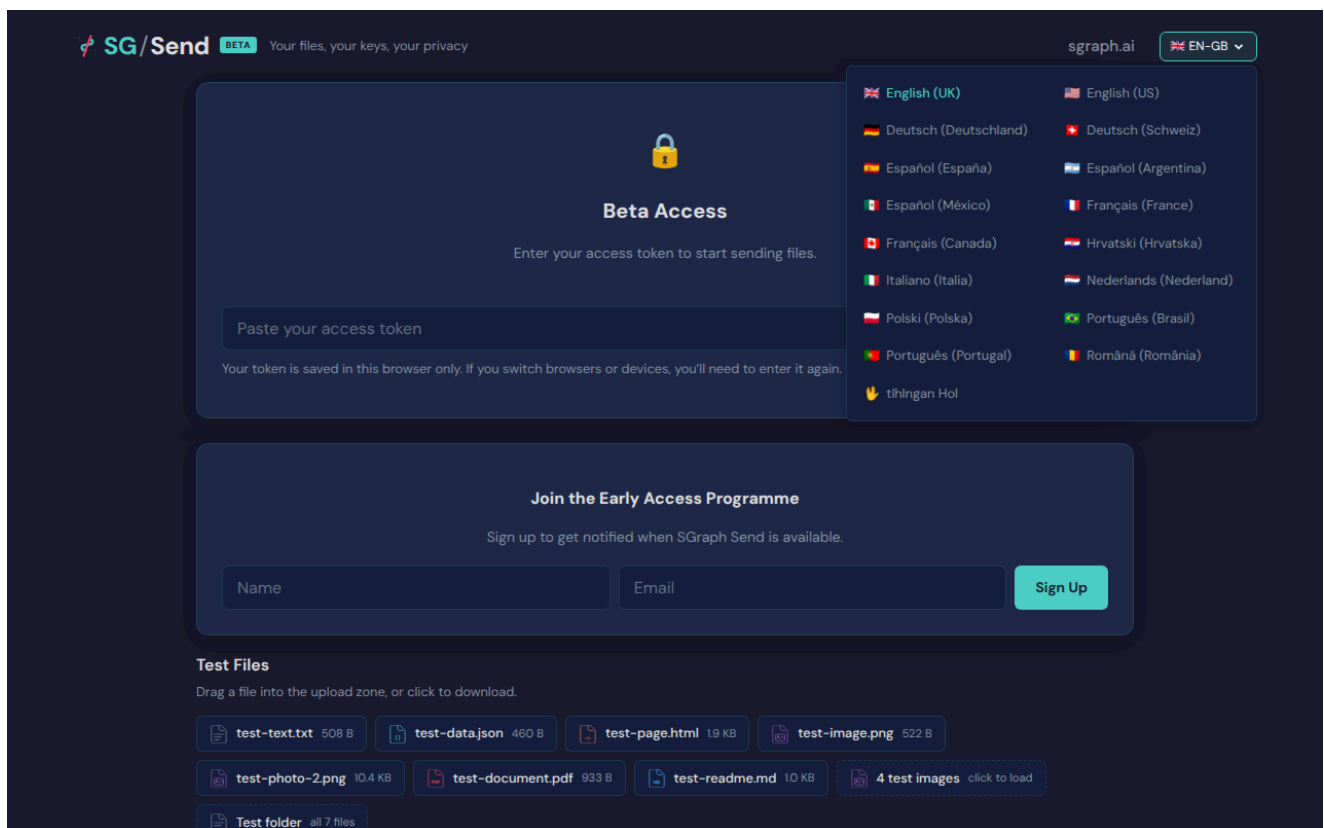
Step 1: Landing page (en-GB)

The SG/Send landing page showing the Beta Access gate. The language selector button ("EN-GB") is visible in the top-right corner.



Step 3: Dropdown open -- all 17 locales visible

This is the key screenshot. The language dropdown is open, showing all 17 supported locales in a two-column grid. This is the same dropdown that causes CR-002 when tests use generic button selectors.



Step 5: After clicking "Deutsch (Deutschland)"

Clicking Deutsch navigated to `http://localhost:10062/de-de/` which returns a 404. This reveals a QA infrastructure finding: `_build_ui_serve_dir` in the `conftest` only copies the `en-gb` locale directory. Non-English locale pages are not available on the local test server.

Error response

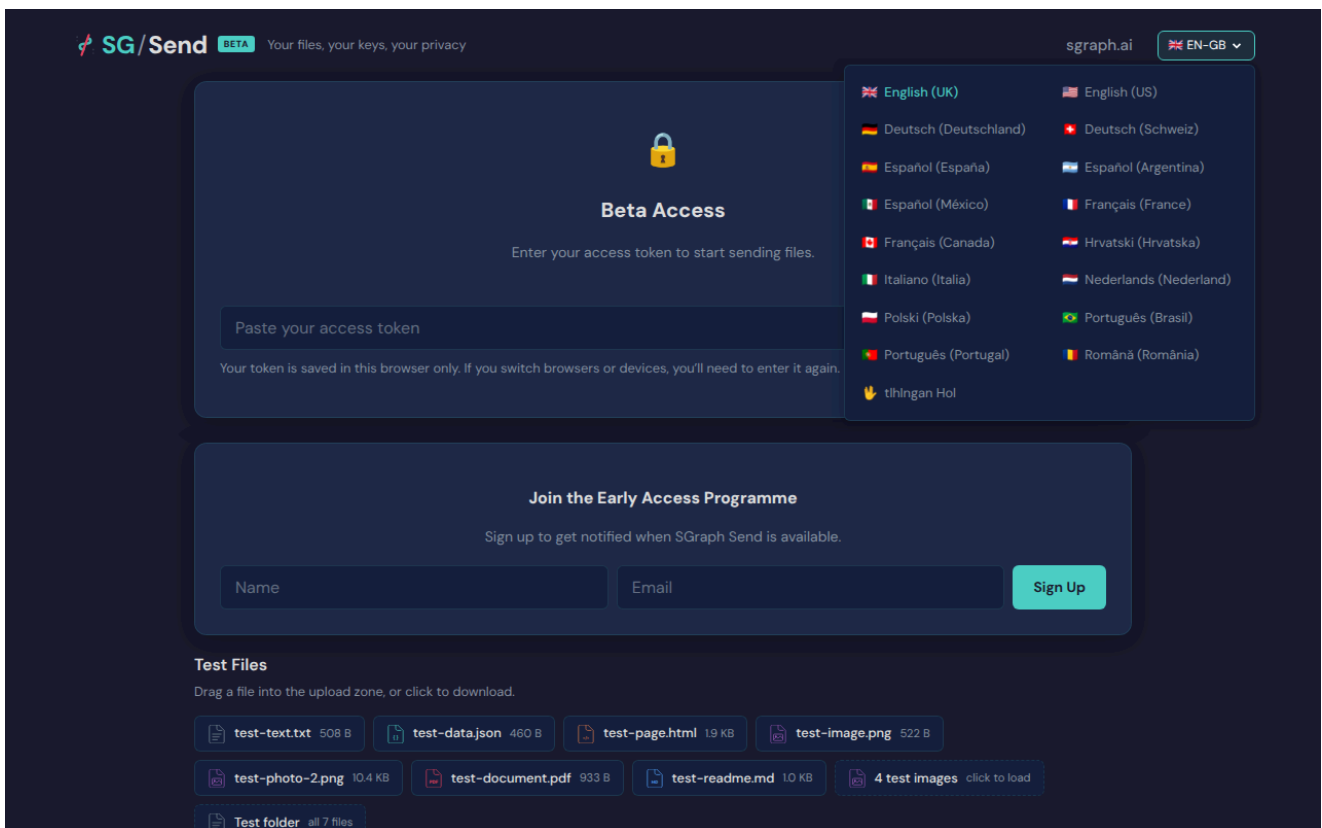
Error code: 404

Message: File not found.

Error code explanation: 404 - Nothing matches the given URI.

Step 10: Locale enumeration (second test method)

The second test method opens the dropdown and checks for 8 sampled locales. All 8 were found: English (UK), English (US), Deutsch (Deutschland), Francais (France), Espanol (Espana), Italiano (Italia), Portugues (Portugal), and tlhIngan Hol (Klingon).



QA Finding from this Test

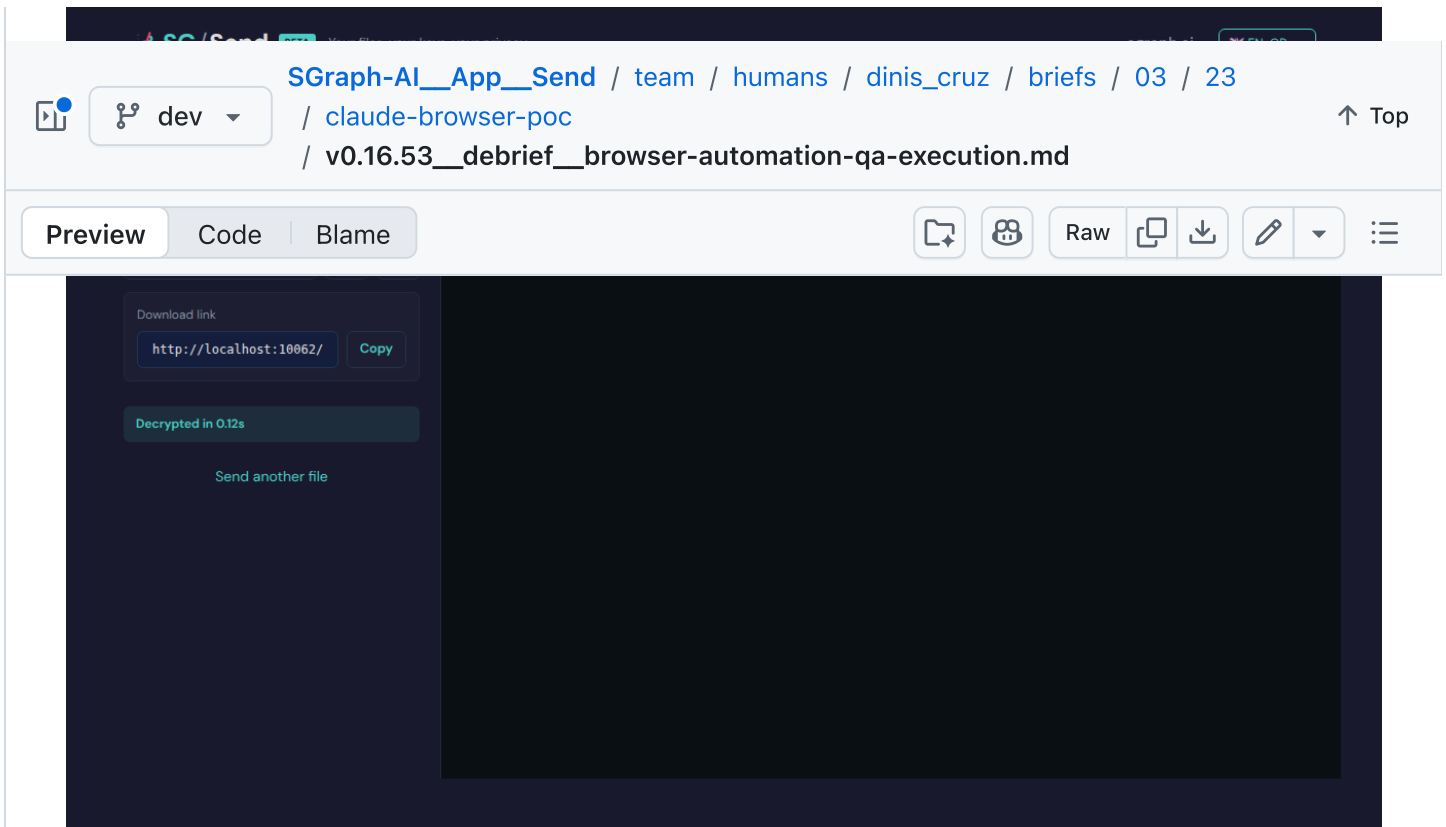
The locale switch to `/de-de/` produces a 404 on the local test server. This is because the QA conftest's `_build_ui_serve_dir` function only copies the `en-gb/` directory from the UI package:

```
# From conftest.py line 96-100
(serve_dir / "en-gb").mkdir(exist_ok=True)
shutil.copy(ui_dir / "en-gb" / "index.html", serve_dir / "en-gb" / "index.html")
for route in ("download", "browse", "gallery", "v", "view", "welcome"):
    src = ui_dir / "en-gb" / route
    ...
```

If i18n testing is desired locally, this function needs to be extended to copy all locale directories. In CI, the i18n pages are generated at build time by `scripts/generate_i18n_pages.py` and deployed to S3, so this only affects the local test stack.

4. Key Screenshot: Combined Link Auto-Decrypt

The most significant screenshot from the full test suite run. This is from `test_combined_link.py::test_combined_link_upload_and_auto_decrypt`:



This proves the complete zero-knowledge pipeline ran through the browser inside this session: file encrypted with AES-256-GCM in the browser, uploaded to the in-memory API, combined link generated, new browser tab opened, JavaScript decrypted client-side, original content displayed. "Decrypted in 0.12s".

5. Known Bug CR-002: Visual Evidence

The access gate test `test_upload_accessible_with_token` fails because submitting the token triggers the language dropdown instead of the Go button. Screenshot evidence:



The language dropdown is open, obscuring the page. The workaround (already in `browser_helpers.py`) is to target `#access-token-submit` by ID instead of using a generic button selector.

6. Deliverables

#	Deliverable	Location
1	This debrief	This document
2	Language selector test	<code>test__language_selector.py</code>
3	Landing page screenshot	<code>screenshots/01_landing_en_gb.png</code>

#	Deliverable	Location
4	Dropdown open screenshot	screenshots/03_dropdown_open.png
5	German 404 screenshot	screenshots/05_switched_to_deutsch.png
6	Locale enumeration screenshot	screenshots/10_all_locales_dropdown.png
7	Combined link auto-decrypt	screenshots/combined_07_auto_decrypted.png
8	CR-002 evidence	screenshots/gate_03_wrong_token_cr002.png

7. What This Enables

An AI agent in a Claude Web session can now:

1. **Start the full SG/Send stack** -- API server + UI server, in-memory, no AWS
2. **Launch a real browser** -- headless Chromium via Playwright
3. **Execute the full QA test suite** -- 103 tests, 92% pass rate
4. **Write new tests** -- create a test file, run it, capture screenshots, report results
5. **Produce living documentation** -- identical screenshot output to the CI pipeline at `qa.send.sgraph.ai`
6. **Discover real bugs** -- the i18n locale limitation and CR-002 evidence were both captured during this session

This means the QA agent can operate independently: verify UI changes, run regression tests, capture visual evidence, and report findings -- all without CI infrastructure, a local machine, or repo write access. Combined with the SGit vault workflow, an agent can build code, test it visually, and push verified patches, closing the loop entirely.